

Dockerfile Instructions — Cheatsheet

The `docker build` CLI command is on the other cheatsheet. This one is about what goes **inside** the `Dockerfile` itself — the instructions that describe how an image gets built, layer by layer.

The core instructions

<code>FROM nginx:alpine</code>	<code># the base image everything else builds on top of – every # Dockerfile must start with one (or a build stage must)</code>
<code>WORKDIR /app</code>	<code># sets the working directory for every instruction after it – # creates the directory if it doesn't exist</code>
<code>COPY package.json .</code>	<code># copy files from the BUILD CONTEXT (your project folder) into # the image – most common instruction in any Dockerfile</code>
<code>ADD https://example.com/file.tar.gz .</code>	<code># like COPY, but can also fetch a URL and auto-extract # .tar archives – prefer COPY unless you need those extras</code>
<code>RUN npm install</code>	<code># execute a command AT BUILD TIME – its result is baked into a # new layer, permanently, in the image</code>
<code>ENV NODE_ENV=production</code>	<code># set an environment variable – persists into the running # container, visible to every process inside it</code>
<code>ARG BUILD_VERSION=1.0</code>	<code># a build-time-only variable – NOT available in the running # container, only while `docker build` is executing</code>
<code>EXPOSE 80</code>	<code># documentation only – declares "this image listens on 80". # Does NOT publish the port; `-p` at `docker run` time does that</code>
<code>VOLUME /data</code>	<code># declares this path should be a volume – Docker will mount an # anonymous volume here automatically if none is specified at run time</code>
<code>USER node</code>	<code># switch from root to this user for all following instructions # AND for the container at runtime – don't run as root unless you must</code>
<code>LABEL maintainer="you@example.com"</code>	<code># arbitrary metadata key/value, visible via `docker inspect`</code>
<code>CMD ["node", "server.js"]</code>	<code># the DEFAULT command when the container starts – overridable at # `docker run <image> <other-command>`</code>
<code>ENTRYPOINT ["node"]</code>	<code># the command that ALWAYS runs – arguments passed at `docker run` # get appended to it, rather than replacing it like with CMD</code>

CMD vs. ENTRYPOINT — the instruction people always mix up

	Overridable at <code>docker run</code> ?	Typical use
<code>CMD ["node", "server.js"]</code> alone	Yes — <code>docker run img bash</code> replaces the whole thing	A sensible default that's fine to override
<code>ENTRYPOINT ["node"]</code> alone	No — args get appended, not replaced	Force the image to always run through one binary
Both together (the common pattern)	<code>ENTRYPOINT</code> is fixed, <code>CMD</code> supplies its <i>default arguments</i> , overridable	<code>ENTRYPOINT ["node"]</code> + <code>CMD ["server.js"]</code> → <code>docker run img other.js</code> runs <code>node other.js</code>

```
ENTRYPOINT ["node"]  
CMD ["server.js"]
```

```
docker run myimage           # runs: node server.js  
docker run myimage other.js  # runs: node other.js  (CMD replaced, ENTRYPOINT stayed fixed)
```

COPY vs. ADD — pick COPY unless you need the extras

	Copies local files	Fetches a URL	Auto-extracts <code>.tar</code> / <code>.tar.gz</code>
<code>COPY</code>	Yes	No	No
<code>ADD</code>	Yes	Yes	Yes

Rule of thumb taught in class: **default to COPY**. `ADD`'s extra behavior is rarely what you actually want, and it's surprising when it's not.

RUN vs. CMD / ENTRYPOINT — build time vs. run time

- `RUN` executes **while the image is being built** — its output is committed as a new layer, permanently. You use it to install packages, compile code, create directories.
- `CMD` / `ENTRYPOINT` don't run at build time at all — they just **record** what should happen when a container is eventually started from this image, later, maybe many times.

```
RUN apt-get update && apt-get install -y curl  # happens ONCE, at build time – baked into the image  
CMD ["curl", "http://example.com"]           # happens EVERY TIME a container starts from this  
image
```

Multi-stage builds — smaller production images

```
# ---- Stage 1: build ----
FROM node:20 AS build
WORKDIR /app
COPY package*.json .
RUN npm install
COPY . .
RUN npm run build          # produces /app/dist

# ---- Stage 2: production ----
FROM node:20-alpine
WORKDIR /app
COPY package*.json .
RUN npm install --omit=dev    # only production dependencies
COPY --from=build /app/dist ./dist # pull ONLY the build output from stage 1
CMD ["node", "dist/server.js"]
```

The whole point: **stage 1's build tools (compilers, dev dependencies, source files) never make it into the final image**. Only `--from=build` gets copied across. This is how you go from a gigabyte-sized image full of build tooling to a lean production image with just the runtime and the compiled output.

`.dockerignore` — the file everyone forgets

Sits next to the `Dockerfile`, same syntax as `.gitignore`. Anything listed here is excluded from the **build context** — never sent to the Docker daemon, never copyable by `COPY` / `ADD`.

```
node_modules
.git
*.log
.env
```

Two real reasons to have one: it keeps the build context small (faster builds), and it stops secrets/local junk (`.env`, `.git`) from ever being copyable into an image layer by accident.

Layer-caching order — the practical habit to teach

Docker caches each layer and reuses it if that instruction and everything before it hasn't changed. **Order instructions from least-often-changed to most-often-changed:**

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json .          # changes rarely
RUN npm install                # so this expensive layer stays CACHED across most rebuilds
COPY . .                      # your actual source – changes on every edit, so put it LAST
CMD ["node", "server.js"]
```

Get this order backwards (`COPY . .` before `npm install`) and **every** source code change invalidates the cache for `npm install` too — turning a 2-second rebuild into a 2-minute one.